OpenCV Getting Started

1. Experiment Objective

Master OpenCV basics: image reading, display, saving, color space conversion, and other basic operations.

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
OpenCV	Open-source computer vision library providing image processing and computer vision algorithms
Image processing	The general term for techniques that analyze, transform, and enhance digital images
Pixel operation	Directly reading and writing the image pixel array to achieve low-level operations such as color modification and region overwriting

2. Technical Architecture

This experiment is a pure offline Python script and does not depend on ROS2 or micro_ros_agent. The script runs in a Linux desktop environment, using the OpenCV library directly to read local images, perform pixel-level operations, and display the results through a GUI window.

Data flow: local image file → `cv2.imread()` loads it as a NumPy ndarray → OpenCV API processing → `cv2.imshow()` displays it → `cv2.waitKey()` waits for user interaction.

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☑ Supported
No-PTZ version	☑ Supported
Required peripherals	None (local image processing, no need to start the car)

2. Startup Method

This experiment provides three running modes, corresponding to different core operations:

Mode 1: Read and display an image

```
python3 ~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_getting_started/opencv_read_show.py
```

Mode 2: Write/save an image

```
python3 ~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_getting_started/opencv_write.py
```

Mode 3: Pixel operations

```
python3 ~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_getting_started/opencv_pixel_manipulation.py
```

3. Operation Instructions

After running, a window pops up to display the image:

- `opencv_read_show.py`: reads and displays the original image, and prints size information to the terminal
- `opencv_write.py`: demonstrates image saving, writing the loaded image to a new file
- `opencv_pixel_manipulation.py`: modifies the pixel values of a specified row to black via NumPy array indexing

Press `q` to close all windows.

4. Stop Method

Press `q` to close the OpenCV window and exit the program.

4. Experiment Phenomena

- Successfully load and display the image
- Understand the usage of `cv2.imread()`, `cv2.imshow()`, and `cv2.imwrite()`

Original image: The test image used for all subsequent OpenCV operation demos, loaded and displayed via `cv2.imread()`.

Result (copied image): Uses `cv2.imwrite()` to save the loaded image data as a new file, verifying the integrity of the image read/write pipeline.

Result (modified image): Directly modifies pixel values via NumPy array indexing (e.g., changing the color of an ROI region), demonstrating that OpenCV image data is essentially a NumPy ndarray.

---

5. Program Analysis

> Source code path:
>
> - Script directory: `~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_getting_started/`
> - Test image: `~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/yahboom.png`

1. Core Logic Analysis

All three scripts follow the same execution flow: **load image → process → display loop → release resources**. The core code of each mode is described below one by one.

Image read and display (opencv_read_show.py)

```
# Source file:
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_getting_started/opencv_read_show.py
import cv2

if __name__ == "__main__":
    # Read the image
    img =
cv2.imread("~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/yahboom.png", cv2.IMREAD_UNCHANGED)
```

```
    # Check whether the image was read successfully
    if img is None:
        print("Image read failed, please check that the images/yahboom.png path
is correct")
        exit()

    print("Image size info:", img.shape)

    while True:
        # Display the image
        cv2.imshow("yahboom", img)

        # Wait for a key event, press 'q' to exit
        action = cv2.waitKey(10) & 0xFF
        if action == ord('q') or action == 113:
            break

    cv2.destroyAllWindows()
```

Key function notes:

- `cv2.imread(path, flag)`: decodes an image file into a NumPy ndarray (BGR format); `IMREAD_UNCHANGED` preserves the original channels
- `img.shape`: returns the `(height, width, channels)` triple, used to get the image dimensions
- `cv2.imshow(winname, img)`: creates and refreshes a GUI window to display the image
- `cv2.waitKey(10)`: waits 10 ms and returns the key ASCII code; it is the core of the OpenCV GUI event loop
- `cv2.destroyAllWindows()`: closes all windows created by OpenCV

Pixel operations (opencv_pixel_manipulation.py)

```
# Source file:
~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/opencv_getting_started/ope
ncv_pixel_manipulation.py
import cv2

if __name__ == '__main__':
    img =
cv2.imread("~/microros_ws/src/microros_v2_opencv_base_pkg/scripts/images/yahboom
.png", cv2.IMREAD_UNCHANGED)
    if img is None:
        print("Image read failed, please check that the images/yahboom.png path
is correct")
        exit()

    (b, g, r) = img[100, 100]
    print(b, g, r)

    # Modify the pixel values of row 10 to black
    i = 10
    for j in range(1, 255):
        img[i, j] = (0, 0, 0)

    while True:
        cv2.imshow("yahboom", img)
```

```
        action = cv2.waitKey(10) & 0xFF
        if action == ord('q') or action == 113:
            break

    cv2.destroyAllWindows()
```

Key logic notes:

- `img[y, x]` returns the `(B, G, R)` triple; OpenCV uses BGR instead of RGB channel order
- `img[i, j] = (0, 0, 0)` directly assigns a value to modify a pixel; `(0,0,0)` is pure black
- The `range(1, 255)` loop sets the first 254 pixels of row 10 to black, forming a black horizontal line

2. Key Parameters

Parameter	Type	Default	Description
`cv2.IMREAD_UNCHANGED`	flag	—	Reads with the original number of channels (including the alpha channel)
Delay of `cv2.waitKey(10)`	int	10ms	GUI refresh interval; too large a value causes the window to stutter

3. Node Notes

This experiment is a pure Python script and does not use ROS2 nodes. The script uses `if __name__ == '__main__':` as its entry point and runs independently in a desktop environment.

6. Common Problems

Problem	Cause	Solution
Image fails to load	The script is not run in the correct directory (the image path is relative)	Make sure the command is executed in the `microros_ws` directory
The window flashes and disappears	Missing waitKey	`cv2.waitKey()` is a mandatory event loop and cannot be omitted